Do it MySQL 입문 5장

조인의 종류

JOIN이란?

- 두 개 이상의 테이블이나 데이터베이스를 연결하여 데이터를 검색하는 방법
- 검색하고 싶은 컬럼이 다른 테이블에 있을 경우 주로 사용
- 여러 테이블을 마치 하나의 테이블인 것처럼 활용

JOIN의 종류

INNER JOIN

- 조인키에 해당하는 각 테이블의 열값을 비교해 조건에 맞는 값을 검색
- 벤다이어그램에서 “교집합”

```
SELECT <col> FROM [table] as A INNER JOIN [table2] as B on
A.col1 = B.col1
WHERE <col> = [value]
ORDER BY [col]

-- A와 B의 col1에 겹치는 행만 가져오기
-- ON문은 조인할 때 조인 조건을 위해서만 사용!
```

OUTER JOIN

- 조인하는 여러 테이블에서 한 쪽 테이블에 데이터가 있고, 다른 쪽에 없는 경우, 데이터가 있는 쪽 테이블의 내용을 전부 출력할 때 사용
- 조건에 만족하지 않아도 해당 행을 출력하고 싶을 때 사용

```
SELECT <col> FROM [table] as A <LEFT, RIGHT, FULL> JOIN [ta
ble2] as B on A.col1 = B.col1
WHERE <col> = [value]
ORDER BY [col]
```

```
-- LEFT JOIN: JOIN 구문 왼쪽 테이블을 기준으로 JOIN
-- RIGHT 테이블에 기준 데이터가 없으면 NULL로 표시
-- RIGHT JOIN: JOIN 구문 오른쪽 테이블을 기준으로 JOIN
-- FULL JOIN: 합집합으로, 양쪽 모두 일치하지 않는 조건까지 모두 결합하여 출력

SELECT <col> FROM [table] as A <LEFT, RIGHT, FULL> OUTER JOIN [table2] as B on A.col1 = B.col1
WHERE <TABLE>.<col> IS NULL
ORDER BY [col]


-- OUTER JOIN: LEFT, RIGHT 중에 "하나에만" 포함된 데이터 출력
```

하지만, MySQL에서는 FULL OUTER JOIN을 지원하지 않음 ⇒ LEFT & RIGHT OUTER JOIN을 합쳐서 사용

CROSS JOIN

- 조인에 포함된 테이블의 cartesian 곱을 산출
- 각 테이블의 모든 경우의 수를 조합한 데이터가 필요한 경우 사용
- FROM 구문에 조인 조건이 없는 것이 특징

```
SELECT <col> FROM [table] as A CROSS JOIN [table2] as B
```

SELF JOIN

- 같은 테이블을 사용하는 특수한 조인
- 테이블의 행을 같은 테이블 안에 있는 다른 행과 조인
- 계층적인 구조를 테이블화 할 경우 사용
- 반드시 테이블에 **Alias 명**을 사용하여 질의

```
SELECT <col> FROM [table] as A <LEFT, RIGHT, FULL> JOIN [table] as B on A.col1 = B.col1
WHERE <col> = [value]
ORDER BY [col]
```

```
-- LEFT JOIN: JOIN 구문 왼쪽 테이블을 기준으로 JOIN
-- RIGHT 테이블에 기준 데이터가 없으면 NULL로 표시
-- RIGHT JOIN:
-- FULL JOIN:
```

쿼리안에 또 다른 쿼리, 서브쿼리

서브쿼리란?

- 쿼리 안에 포함되어 있는 또 다른 쿼리
- 조인하지 않은 상태에서 다른 테이블과 일치하는 행을 찾거나, 조인 결과를 다시 조인할 때 사용
- 서브쿼리의 특징
  - 반드시 소괄호로 감싸 사용
  - 주쿼리를 실행하기 전에 1번만 실행
  - 비교 연산자에 서브쿼리를 사용하는 경우 서브쿼리를 오른쪽에 기술
  - 서브쿼리 내부에는 정렬구문인 ORDER BY 문 사용 불가

```
SELECT a.film_id, a.title, a.special_features, x.name
FROM film AS a
INNER JOIN (
SELECT b.film_id, c.name
FROM film_category as b
INNER JOIN category as c ON b.category_id = c.category_id
WHERE b.film_id > 10 AND b.film_id <20) AS x ON a.film_id =
x.film id
```

스칼라 서브 쿼리

데이터베이스 성능에 영향을 줘서 사용을 자제하는 게 좋음..!

```sql
SELECT a.film_id, a.title, a.special_features,
(SELECT c.name FROM film_category as b INNER JOIN category
AS c ON b.category_id = c.category_id
WHERE a.film_id = b.film_id) as name
FROM film as a
WHERE a.film_id >10 AND a.film_id <20;
```

공통 테이블 표현식

책에서만 언급!